KLA FLEETPACK · NEXUS THEME V2

FleetPack FAM

Fleet Availability Module — Complete Technical Documentation

Folder Structure · Architecture · Base Module · Shared Module · Core · Features

---

Angular 20 · Signals · Tailwind CSS 4 · Chart.js 4 · Mock REST API

Generated documentation — consolidates README, component reference and architecture guide

Table of Contents

1. Overview & Quick Start
2. Full Folder Structure
3. End-to-End Data Flow
4. The Base Module — Reusable Component Library
5. The Shared Module — App-Level Composition Layer
6. The Core Module — Data & Infrastructure Layer
7. Layout — The App Shell
8. Features & Routing
9. Reference — Domain Models
10. Adding a New Module — Checklist

1. Overview & Quick Start

Angular implementation of the **FAM (Fleet Availability Module)** screens from the Nexus Theme v2 design (KLA FleetPack). Standalone components, signal-based state management, Chart.js visualizations, custom heatmap/Gantt renderers, and a fully mocked REST API — no backend required.

```
npm install
npm start          # ng serve --open → http://localhost:4200
```

Requires Node 18.19+, 20.11+, or 22.0+ (Angular 20). Stack: Angular 20 + @angular/cdk · Tailwind CSS 4 (CSS-first `@theme` tokens) · Chart.js 4 + ng2-charts 6 · RxJS 7 · signals for all state (no NgRx/Akita).

Screens & Routes

Route	Screen
`/fleet-availability/up-time/analysis`	**Fleet Up+Time Analysis** — **Uptime/Downtime Trend toggle** on the rolling line chart (1W/13W + period average), **collapsible** uptime breakdown table (rolling, tool-wise, grouped by SW version) with **chip-style Tool-wise/Grouped toggles** + CSV export, Top-10 unavailable tools stacked bar, downtime category details with **colored progress bars**.
`/fleet-availability/up-time/availability`	**Fleet Up+Time Availability** — **single divided KPI strip** (13W/4W rolling, current week, MTBr, total downtime in red), Uptime/Downtime Trend line chart, Top-10 unavailable stacked bar with a **Non-Scheduled toggle**, downtime categories (colored label + underline bar), a collapsible **Tool-Level Analysis Filter** bar (tool picker + live state-count pills).
`…/availability/heatmap`	Tab 1 — **24-Hour State Heatmap** (custom CSS-grid, 14 days × hour blocks; state totals now live in the page-level Tool-Level Analysis Filter bar, not the heatmap header).
`…/availability/gantt`	Tab 2 — **Activity Gantt** (Sys E10 / Tool E10 rows per day, segment labels, **colored availability badges**, "Day Shift" as a legend swatch — the old day-shift mask toggle was removed — pagination in the footer next to the legend).
`…/availability/events`	Tab 3 — **Event Details** (grouped-by-day table with **sortable timestamps**, **progress-bar duration**, **outlined state badges**, **delete/edit row actions**, paging, Add Event, CSV).
`/alarm-explorer`	**Alarm Explorer home** — **Model/Fleet/Category/Duration filter bar**, stacked alarm-volume chart by category (Trend/Pareto tabs), fleet breakdown drill-down list.
`/alarm-explorer/fleet/:fleetId`	**Fleet detail** — **Alarm ID/Tools/Category/SW Version filter bar**, **5-metric KPI strip**, tool-level stacked distribution (Trend/Pareto tabs, click a bar to drill down), top alarms with **category-colored bars**, tool summary table.
`/alarm-explorer/fleet/:fleetId/tool/:toolId`	**Tool alarm summary** — **Alarm ID/Category/Severity/Recipe filter bar** + searchable alarm table; selecting a row opens the **Alarm Info** inspector in a **slide-over <base-drawer>** (occurrence stats, recipe context, weekly-trend chart) — it no longer renders as a permanent inline side panel.
`…/tool/:toolId/alarm/:alarmId`	**Alarm events** — **Date Range/Recipe/Show/Duration filter bar**, **4-metric KPI strip**, chronological event log with a **recipe tab filter**, outlined resolution badges, pagination, CSV.
`/fleet-configuration`, `/fleet-productivity`, `/tqual`, `/my-reports`, `/innovation-lab`, `/engineering-utilities`	Placeholder modules wired into nav & routing.

Layering Rule

`features/` depends on `shared/` and `core/`; `shared/` depends on `base/` and `core/`; `base/` depends on nothing app-specific (no chart.js, no models, no services) — it is a self-contained, exportable design-system library.

2. Full Folder Structure

```
fleetpack-base-module/
├── angular.json             Angular CLI workspace config
├── package.json             deps: @angular/*, chart.js, ng2-charts, rxjs, tailwindcss
├── tsconfig.json / tsconfig.app.json
├── postcss.config.js / .postcssrc.json   Tailwind 4 via PostCSS
├── README.md                top-level quick start + architecture summary
├── docs/
│   └── FleetPack-FAM-Documentation.pdf   this document
└── src/
    ├── main.ts              bootstrapApplication(AppComponent, appConfig)
    ├── index.html
    ├── styles.css           Tailwind 4 @theme tokens + component classes (panel, btn-*, table-th, chip-toggle, …)
    └── app/
        ├── app.component.ts       root shell — just <router-outlet>
        ├── app.config.ts          providers: router, http (+interceptors), charts, widget bootstrap
        ├── app.routes.ts          route tree (see §8)
        │
        ├── base/                     REUSABLE COMPONENT LIBRARY (design-system layer)
        │   ├── index.ts                          barrel export — public API of the module
        │   ├── models/table.model.ts              BaseColumnDef, BaseCellKind, sort/page/filter/click events
        │   ├── directives/cell-template.directive.ts   [baseCell] custom-cell-template directive
        │   └── components/
        │       ├── base-table.component.ts        <base-table> — the core data grid
        │       ├── base-paginator.component.ts     <base-paginator>, <base-search-input>
        │       ├── base-ui.components.ts           <base-badge>, <base-trend>, <base-kpi-card>,
        │       │                                    <base-loading>, <base-empty-state>, <base-sparkline>
        │       ├── base-form.components.ts         <base-button>, <base-text-input>, <base-textarea>,
        │       │                                    <base-select>, <base-checkbox>, <base-radio-group>, <base-toggle>
        │       ├── base-datepicker.component.ts    <base-datepicker>
        │       ├── base-nav.components.ts          <base-breadcrumbs>, <base-tabs>, <base-dropdown-menu>
        │       ├── base-overlay.components.ts      <base-modal>, [baseTooltip], <base-alert>,
        │       │                                    <base-progress-bar>, <base-skeleton>
        │       └── base-drawer.component.ts        <base-drawer> — slide-over side panel (NEW)
        │
        ├── shared/                   APP-LEVEL COMPOSITION LAYER (built ON TOP of base/)
        │   ├── components/ui.components.ts   fam-kpi / fam-loading / fam-trend (deprecated
        │   │                                 thin wrappers over base-*) + fam-state-legend
        │   ├── dynamic/                       the dynamic/config-driven widget system
        │   │   ├── widget.model.ts                    WidgetConfig union (kpi-grid/chart/table/ranked-list/component)
        │   │   ├── dynamic-page.component.ts          <fam-dynamic-page>, <fam-dynamic-widget> (grid + chrome)
        │   │   ├── basic-widgets.components.ts        KPI grid / chart / ranked-list renderers
        │   │   ├── table-widget.component.ts          table renderer — adapter onto <base-table>
        │   │   ├── widget-registry.ts                  name → component Map (registerWidget/resolveWidget)
        │   │   └── register-widgets.ts                 bootstraps built-in named widgets (heatmap, gantt, …)
        │   └── utils/
        │       ├── csv.util.ts               downloadCsv() — CSV export helper
        │       └── category-colors.util.ts   buildCategoryColorMap / buildCategoryTextColorMap (NEW)
        │
        ├── core/                     DATA / INFRASTRUCTURE LAYER
        │   ├── models/models.ts                   every API response/row TypeScript interface
        │   ├── mock-api/
        │   │   ├── mock-data.ts                   deterministic build...() fake-data generators (seeded PRNG)
        │   │   └── mock-api.interceptor.ts        routes /api/** → a generator, adds fake latency
        │   ├── services/api.service.ts            ApiService — one typed HttpClient method per endpoint
        │   ├── state/                         signal-based state management (see §6.4)
        │   │   ├── base.store.ts               createQuery / createPagination / createListFilter primitives
        │   │   ├── filter.store.ts             FilterStore — global fleet/date/duration filter
        │   │   ├── uptime.store.ts             UptimeStore — Up-Time Analysis + Availability
        │   │   ├── alarm.store.ts              AlarmStore — Alarm Explorer drill-down
```

```
│   │   └── segment-derivation.util.ts  derives Gantt bars + Event rows from one raw feed
│   ├── auth/
│   │   ├── auth.service.ts             AuthService — signal-based session, dummy JWT in localStorage
│   │   ├── auth.guard.ts               authGuard / authChildGuard (CanActivateFn)
│   │   └── auth.interceptor.ts         attaches Bearer token to outgoing requests
│   └── constants/
│       ├── app.constants.ts            routes, nav groups, brand text, auth config, UI copy
│       └── component-catalog.ts        COMPONENT_CATALOG — runtime index of every component
│
├── layout/               App shell (persistent frame around every routed feature)
│   ├── shell.component.ts             <fam-shell> — sidebar + topbar + <router-outlet> + footer
│   ├── sidebar.component.ts           <fam-sidebar> — nav, grouped by NAV_GROUPS
│   └── topbar.component.ts            <fam-topbar> — breadcrumbs, Date Range/fleet/duration filters, user
│
└── features/              Screens = header + WidgetConfig[] (thin, no data logic)
    ├── auth/login.component.ts
    ├── uptime-analysis/uptime-analysis.component.ts
    ├── uptime-availability/
    │   ├── uptime-availability.component.ts   page shell + routed child tabs + Tool-Level Analysis Filter bar
    │   ├── state-heatmap.component.ts           registered widget: 'state-heatmap'
    │   ├── activity-gantt.component.ts          registered widget: 'activity-gantt'
    │   ├── event-details.component.ts           registered widget: 'event-details'
    │   └── segment-activities.component.ts
    ├── alarm-explorer/
    │   ├── alarm-home.component.ts
    │   ├── fleet-detail.component.ts
    │   ├── tool-alarms.component.ts              hosts the <base-drawer> alarm inspector
    │   ├── alarm-events.component.ts
    │   └── alarm-info-panel.component.ts         rendered inside tool-alarms' drawer (no longer a registered widget)
    ├── placeholder/placeholder.component.ts     "coming soon" screen for unbuilt modules
    └── dev/
        ├── component-catalog.component.ts       renders COMPONENT_CATALOG, route /dev/components
        └── base-playground.component.ts         live demo of every base-* component, route /dev/base
```

3. End-to-End Data Flow

```
Component (features/**)
   │  reads signals from, calls actions on
   ▼
Store (core/state/*.store.ts — createQuery/createPagination/createListFilter)
   │  calls
   ▼
ApiService (core/services/api.service.ts — typed HttpClient wrapper)
   │  HTTP GET /api/**
   ▼
authInterceptor → mockApiInterceptor (core/mock-api/mock-api.interceptor.ts)
   │  matches path → generator, adds latency
   ▼
mock-data.ts (build...() — deterministic seeded fake data)
```

A component never talks to `HttpClient` or mock data directly — only to a store. That indirection is what makes the mock-to-real-backend swap a one-file change (remove `mockApiInterceptor` from `app.config.ts`).

For rendering, a component composes `WidgetConfig[]` from store data and hands it to `<fam-dynamic-page>`, which dispatches each widget to a renderer built from `base/` primitives. Custom, feature-specific panels (heatmap, gantt, event details) are registered by name so they can be embedded as `{ type: 'component', name: '...' }` widgets inside any dynamic page. The Alarm Explorer's inspector panel is the one exception: it's imported directly and rendered inside a `<base-drawer>` by its host page, since a drawer overlays the whole viewport rather than sitting inside the page's widget grid.

Layer	File(s)	Responsibility
Models	`core/models/models.ts`	Typed response/row shapes — the contract between mock and real API
Mock data	`core/mock-api/mock-data.ts`	`build...()` generators producing deterministic fake payloads
Mock routing	`core/mock-api/mock-api.interceptor.ts`	Matches `/api/**` paths to a generator, adds artificial latency
Service	`core/services/api.service.ts`	One typed method per endpoint, returns `Observable<T>`
Store	`core/state/*.store.ts`	Signal-based state: loading/error/data, caching, pagination
Component	`features/**`	Template + `computed()` widget configs, no data-fetching logic

4. The Base Module — Reusable Component Library `base/`

Self-contained, dependency-free (no chart.js required) set of standalone Angular components. Everything uses **signal inputs** (props) and **typed outputs** (event listeners), `OnPush` change detection, and the new control-flow syntax. Live demo route: `/dev/base`.

Everything is re-exported from one barrel, `src/app/base/index.ts` — always import via `import { ... } from '../../base'`, never reach into a component file directly.

4.1 Components

Selector	Purpose
`<base-table>`	Core data table: dynamic columns, pagination, custom cell templates, filters, sticky header + sticky columns, sorting, selection, grouped rows
`<base-paginator>`	Standalone pagination control (also used inside the table)
`<base-search-input>`	Debounced search box
`<base-kpi-card>`	KPI metric card with optional trend + click event
`<base-badge>`	Status pill
`<base-trend>`	▲/▼ percentage pill
`<base-sparkline>`	Inline SVG mini chart (also the table's `sparkline` cell kind)
`<base-loading>`	Spinner row
`<base-empty-state>`	Empty placeholder with optional action button
`<base-button>`	Button with variants, sizes, loading state
`<base-text-input>`	Text/number/password input: label, hint, error, prefix/suffix, clearable
`<base-textarea>`	Multiline input with character counter
`<base-select>`	Custom dropdown with optional search; `[showChevron]="false"` **(NEW)** renders it as a plain input-styled box
`<base-checkbox>` / `<base-radio-group>` / `<base-toggle>`	Choice controls
`<base-datepicker>`	Popup calendar: min/max, disabled-date rule, clearable
`<base-breadcrumbs>`	Navigation trail (routerLink or click events)
`<base-tabs>`	Headless tab bar (underline or pills)
`<base-dropdown-menu>`	Actions menu with icons, dividers, danger items
`<base-modal>`	Content-projected dialog with footer slot
`<base-drawer>`	**NEW —** content-projected slide-over panel: left/right `side`, configurable `width`, backdrop dim, ESC/backdrop-to-close, footer slot
`[baseTooltip]`	Tooltip directive for any element
`<base-alert>`	Info/success/warning/error banner
`<base-progress-bar>`	Progress with label
`<base-skeleton>`	Loading placeholder

All form controls expose a two-way `[(value)]` / `[(checked)]` model and implement `ControlValueAccessor`, so they also work with `ngModel` and Reactive Forms (`formControlName`) out of the box.

```
<base-text-input label="Tool ID" [(value)]="toolId" [clearable]="true"
                 (enterPressed)="search($event)" />
<base-select label="Fab" [options]="fabOptions" [(value)]="fab" [searchable]="true" />
<base-datepicker label="Maintenance" [(value)]="date" [min]="minDate"
                 [disabledDates]="noWeekends" />
<base-breadcrumbs [items]="crumbs" (itemClick)="onCrumb($event)" />
<base-tabs [tabs]="tabs" [(activeId)]="active" />
<base-modal [(open)]="show" title="Edit"> ... <div footer>...</div> </base-modal>
<!-- NEW -->
<base-drawer [(open)]="showInspector" side="right" width="460px" [showClose]="false">
  <fam-alarm-info-panel />
  <div footer class="w-full flex gap-2">
    <button class="btn-primary flex-1">View Event Log</button>
    <button class="btn-ghost flex-1 border border-slate-200">Export</button>
  </div>
</base-drawer>
```

Content projection gotcha: anything passed to `<base-drawer>`'s `[footer]` slot must be a *direct child* of `<base-drawer>` in the host template. Angular content projection does not reach through a nested component's own template — so footer buttons cannot live inside `<fam-alarm-info-panel>` itself if that component is what gets projected as the drawer's body.

4.2 Table Model (models/table.model.ts)

All types consumed by `<base-table>`. A table is fully described by `BaseColumnDef[]` (dynamic columns, add/remove/reorder at runtime) and `rows: T[]` (any row shape).

Cell kind	Renders
`text`	default — `String(value)`
`number`	right-aligned, formatted via `Intl.NumberFormat`
`date`	formatted via `Intl.DateTimeFormat`
`badge`	pill, colored via `badgeClassMap`
`dot`	status dot + text, colored via `dotClassMap` (dot is omitted for values with no map entry)
`trend`	▲/▼ % pill (BaseTrendComponent)
`image`	thumbnail `<img>`, value = URL
`progress`	0–100 progress bar; bar width honors `progressMax`, any positive value gets a minimum visible sliver
`text-bar` **(NEW)**	colored label (via `classFn`) with a thin proportional bar underneath, driven by `barValue` when it differs from the cell's own value
`sparkline`	mini line chart, value = `number[]`
`link`	anchor; href from `linkHref(row)`
`row-actions` **(NEW)**	small icon buttons per row (e.g. delete/edit); `variant: 'button'` renders a bordered ghost button instead of a plain icon
`template`	custom `ng-template` (BaseCellDirective)

Row grouping (`groupBy`) supports three header styles via `groupHeaderStyle`: `'accent'` (default — indigo header with a row count), `'plain'` (muted uppercase section divider, no count), and `'light'` **(NEW)** (white header, bold label, a rounded count pill labelled via `groupCountLabel`, and a solid action button). Return `null` from `groupBy` to leave a row ungrouped — it renders with no header row at all.

Quick start — table

```
import { BaseTableComponent, BaseCellDirective, BaseColumnDef } from '../../base';

columns: BaseColumnDef<Tool>[] = [
  { key: 'toolId', header: 'Tool', sticky: 'left', width: '120px', sortable: true, filterable: true },
  { key: 'photo',  header: 'Photo', kind: 'image' },
  { key: 'uptime', header: 'Uptime %', kind: 'number', align: 'right', sortable: true },
  { key: 'status', header: 'Status', kind: 'badge',
    badgeClassMap: { UP: 'bg-emerald-50 text-emerald-600', DOWN: 'bg-red-50 text-red-600' } },
  { key: 'history', header: '7-day', kind: 'sparkline' },
  { key: 'actions', header: '', sticky: 'right', width: '90px' } // custom template below
];
```

```
<base-table class="panel block"
  [columns]="columns" [rows]="rows" trackKey="toolId"
  [showFilterRow]="true" [stickyHeader]="true" maxHeight="420px" minWidth="1100px"
  selectable="multiple"
  (rowClick)="open($event.row)"
  (sortChange)="onSort($event)"
  (pageChange)="onPage($event)"
  (filterChange)="onFilter($event)"
  (selectionChange)="selected = $event">

  <!-- CUSTOM CELL TEMPLATE: any content — text, images, charts, buttons -->
  <ng-template baseCell="actions" let-row>
    <button class="btn-ghost" (click)="edit(row)">Edit</button>
  </ng-template>
</base-table>
```

Server-side mode

Set `[serverSide]="true"` and `[totalItems]="totalFromApi"`. The table stops filtering/sorting/paginating internally and only emits `(filterChange)`, `(sortChange)`, `(pageChange)` — fetch data in the host and pass the new page via `[rows]`.

Sticky columns

Set `sticky: 'left' | 'right'` and a fixed `width` on the column def. Pinned columns are automatically ordered to the edges and offsets are computed.

4.3 [baseCell] Custom Cell Template Directive

Attach to an `<ng-template baseCell="columnKey">` to fully replace a column's rendering (text, images, charts, buttons, anything). Exposes a typed template context: `$implicit` (row), `value`, `column`, `index`. A template always wins over the column's built-in `kind`.

4.4 Used Throughout the App

The whole application renders through this module — treat the features as live examples:

- **All dynamic tables** (Uptime Analysis, Availability events/activities, Alarm Explorer) → `<base-table>` + `<base-paginator>` via the `fam-table-widget` adapter, incl. grouped rows, group actions, highlight.
- **KPI grids / ranked lists** → `<base-kpi-card>`, `<base-trend>`, `<base-progress-bar>`.
- **Top bar & page filter bars (expanded)** — the global topbar plus new local filter bars (Alarm Explorer's Model/Fleet/Category/Duration, Fleet detail's Alarm ID/Tools/Category/SW Version, Alarm Events' Date Range/Recipe/Show/Duration, Uptime Availability's Tool-Level Analysis Filter) → `<base-breadcrumbs>` route trail + `<base-select>` filters, several with `[showChevron]="false"` for a plain-input look.
- **Login** → `<base-text-input formControlName>` (ControlValueAccessor), `<base-button>`, `<base-alert>`.
- **Alarm Explorer pages (updated)** → `<base-breadcrumbs>` trails; the Tool page uses `<base-search-input>`, `<base-select>`, `<base-button>`, and opens its alarm inspector in a `<base-drawer>` instead of an inline side panel.

- `fam-kpi` / `fam-loading` / `fam-trend` are **deprecated wrappers** delegating to base — new code should import from `src/app/base` directly.

5. The Shared Module — App-Level Composition Layer `shared/`

Purpose: everything the app needs that isn't a generic reusable primitive — glue code, the dynamic/config-driven page system, and FAM-domain widgets. Built on top of `base/`, never the other way around.

5.1 components/ui.components.ts

- `KpiComponent` (`fam-kpi`), `LoadingComponent` (`fam-loading`), `TrendPillComponent` (`fam-trend`) — deprecated thin wrappers that delegate 100% of rendering to `<base-kpi-card>` / `<base-loading>` / `<base-trend>`. Kept only so older feature templates keep compiling; new code should import the `base-*` component directly.
- `StateLegendComponent` (`fam-state-legend`) — a domain-specific legend for the machine states, colored from the FAM Tailwind theme tokens (`bg-state-*`). Accepts `withGap` and **the new** `withDayShift` flags to append the Gap / Day Shift swatches used by the heatmap and Activity Gantt.

5.2 dynamic/ — the config-driven widget system

Core idea: every screen is data (`WidgetConfig[]`), not a hand-written template. A feature component builds `computed<WidgetConfig[]>` and renders one line: `<fam-dynamic-page [widgets]="widgets()" />`. Adding a panel to a screen means adding an object to that array, not writing new HTML.

type	Interface	Renders
`kpi-grid`	`KpiGridWidget`	Auto-fit grid of `<fam-kpi>` tiles
`chart`	`ChartWidget`	Chart.js canvas (chartType, data, options), optional `onPointClick` drill-down; legend renders below the canvas **(moved from the header)**
`table`	`TableWidget<Row>`	Sortable columns (text/mono/badge/dot/trend/progress/**text-bar/row-actions** cell kinds), row grouping (**accent/plain/light**), pagination, row actions
`ranked-list`	`RankedListWidget`	Ranked rows with title/subtitle/value/trend pill/progress bar (**per-item barColor, custom title/subtitle classes**), clickable
`component`	`ComponentWidget`	Arbitrary Angular component, via direct `component: Type<...>` or `name` + registry

All widget types share `WidgetBase` : `id` (for `@for` tracking), `title` , `titlePrefix` **(NEW — muted text before the title)**, `subtitle` , `badge` , `colSpan` (1–6, on a 6-column grid), `legend` , `actions` (buttons in the panel header — pass `active: boolean` on an action to render it as a segmented toggle chip instead of a plain button), `actionsLabel` / `note` (muted labels around the actions row), and `frameless` (skip the panel chrome entirely). New in this revision:

- `dateRange: { from, to }` — a 2-line muted date range shown top-right of the header.
- `tabs: { items, activeId, onChange }` — a segmented pill switch in the header (e.g. Trend/Pareto, Uptime/Downtime Trend).
- `toggle: { label, checked, onChange }` — a single boolean switch in the header (e.g. "Non-Scheduled", "Period").
- `search: { placeholder, value, onChange }` — an inline search box in the header.
- `collapsible` / `collapsed` / `onToggleCollapse` — an accordion chevron before the title; the body (and subtitle) hide when collapsed.

`dynamic-page.component.ts` — `DynamicPageComponent` (`<fam-dynamic-page>`): responsive `grid-cols-1 xl:grid-cols-6` layout; each widget's `colSpan` maps to an `xl:col-span-N` class. `DynamicWidgetComponent` (`<fam-dynamic-widget>`): renders the shared panel chrome (badge/title/date-range/tabs/toggle/ search/actions, described above) then `@switch (widget.type)` -dispatches to the concrete renderer. For `type: 'component'` it resolves the component class via `resolveWidget(name)` and renders it with `NgComponentOutlet` , passing `inputs` .

`widget-registry.ts` — a plain `Map<string, Type<unknown>>` with `registerWidget(name, component)` / `resolveWidget(name)` / `registeredWidgets()` . Lets a `component` -type widget reference a component by string name, so a screen (or a future server-driven page config sent from a real backend) can embed a bespoke panel without importing its class.

`register-widgets.ts` — `registerBuiltInWidgets()` , called once at bootstrap via `provideAppInitializer` in `app.config.ts` . Registers `state-heatmap` , `activity-gantt` , `event-details` , and (still registered, though no longer resolved this way) `alarm-info-`

`panel`. **The alarm inspector is now imported directly by `ToolAlarmsComponent` and rendered inside its `<base-drawer>`**, since drawer content sits outside the page's widget grid rather than inside it.

`table-widget.component.ts` — `TableWidgetComponent` (`fam-table-widget`): an adapter, not an independent implementation. It maps the legacy `ColumnDef` / `TableWidget` shape onto `BaseColumnDef` / `<base-table>` + `<base-paginator>`, so all actual table rendering (cells, grouping, selection highlight, sticky columns, paging UI) is delegated to `base/`.

5.3 utils/

- `csv.util.ts` — `downloadCsv(filename, rows)`: escapes/quotes cell values, builds a CSV Blob, and triggers a browser download via a synthetic `<a download>` click. Used by every screen with a CSV export button.
- `category-colors.util.ts` **(NEW)** — `buildCategoryColorMap(categories)` and `buildCategoryTextColorMap(categories)`: assign a stable Tailwind background/text color to each distinct category, in first-seen order, from two parallel literal palettes (kept literal, not derived by string replacement, so Tailwind's build-time class scanner can see every class name). Used by the downtime-category and alarm-category color coding across Up-Time and Alarm Explorer screens.

6. The Core Module — Data & Infrastructure Layer `core/`

6.1 models/models.ts

Every TypeScript interface that is the contract between mock data and (eventually) a real backend — grouped by feature area: Global (`GlobalFilters` , `ToolState` , `AlarmCategory` , `Severity`), Up-Time Analysis (`WeeklyUptimePoint` , `UptimeAnalysisResponse` , ...), Up-Time Availability (`AvailabilityResponse` , `GanttDay` / `GanttSegment` , `HeatmapDay` , `StateSegment` , `SegmentActivity` , ...), Alarm Explorer (`AlarmHomeResponse` , `FleetAlarmDetailResponse` , `AlarmDefinition` , `AlarmEvent` , ...). Some interfaces intentionally mirror the upstream API's exact casing (e.g. `StateSegment.ToolDetail` , `UptimeInfoPoint.GranulariReferencePoint`) since they are meant to be a drop-in match for a real backend contract.

> **Updated this revision:** `AvailabilityResponse.topUnavailable` now carries a full `unscheduledHrs` / `scheduledHrs` / `nonScheduledHrs` breakdown per tool (previously just a total `hrs`), and `FleetAlarmDetailResponse.topAlarms` items now carry a `category: AlarmCategory` field — both feed the newly-added stacked/colored bar visuals.

6.2 mock-api/

- `mock-data.ts` — `build...()` generator functions producing deterministic fake payloads, seeded with a `mulberry32` PRNG so results are stable across reloads (no flicker between renders, reproducible screenshots). `StateSegment[]` (raw state timeline) and `SegmentActivity[]` (recipe/task detail) are generated once and correlated, so the Gantt chart, heatmap, and Event Details table all derive from the same underlying feed instead of being mocked independently.
- `mock-api.interceptor.ts` — `mockApiInterceptor: HttpInterceptorFn` . Intercepts any request whose path starts with `/api/` , pattern-matches the remaining path segments, calls the matching `build...()` generator, and responds with `delay(280–600ms)` to simulate network latency. Registered in `app.config.ts` via `provideHttpClient(withInterceptors([authInterceptor, mockApiInterceptor]))` . To go live: remove `mockApiInterceptor` from that array — no component changes needed.

Endpoints: `GET /api/fleets` , `/api/uptime/analysis` , `/api/uptime/trend` , `/api/uptime/availability` , `/api/tools/state-segments` , `/api/tools/segment-activities` , `/api/alarms/home` , `/api/alarms/fleets/:fleetId` , `/api/alarms/fleets/:fleetId/tools/:toolId` , `/api/alarms/fleets/:fleetId/tools/:toolId/alarms/:alarmId/events` .

6.3 services/api.service.ts

`ApiService` (`providedIn: 'root'`) — the single HTTP gateway. One typed method per endpoint above, each returning `Observable<ResponseType>` via `HttpClient.get` . Stores are the only consumers of this service; feature components never inject it directly for data (only `TopbarComponent` calls `getFleets()` to populate the fleet picker).

6.4 state/ — signal-based stores

`base.store.ts` — three composable primitives every feature store is built from (no NgRx/Akita; hand-rolled signals):

- `createQuery(fetcher, { cacheTtlMs, keepPreviousData })` — wraps an Observable-returning API call as `{ data, status, loading, loaded, error, lastParams, load(...params), refresh(), clear() }` . Handles: TTL cache keyed by params, in-flight de-dupe, cancellation of a superseded request, and `refresh()` (bypasses cache).
- `createPagination(sourceSignal, pageSize)` — client-side paging over any `Signal<T[]>` : `{ page, pageSize, total, pageCount, paged, next, prev, goTo, reset, setPageSize }` .
- `createListFilter(sourceSignal, initial, predicate)` — reactive predicate filtering: `{ criteria, filtered, setCriteria }` .

```
@Injectable({ providedIn: 'root' })
export class MyModuleStore {
  private api = inject(ApiService);
  readonly detailQuery = createQuery(
    (id: string) => this.api.getDetail(id),
    { cacheTtlMs: 300_000 }
  );
  readonly rows = computed(() => this.detailQuery.data()?.rows ?? []);
```

```
  readonly pager = createPagination(this.rows, 20);
}
```

`filter.store.ts` — `FilterStore`: the global cross-module filter (`fleet`, `dateFrom`/`dateTo`, `duration`), plus the fleet list (populated by `TopbarComponent` on init). Any store can react to `filters()`/`fleet()` changing via an Angular `effect`. **The topbar now renders Date Range first, then Fleet, then Duration** (previously Fleet/Duration/Date Range).

`uptime.store.ts` — `UptimeStore`: backs both Up-Time Analysis and Availability screens. Five `createQuery` instances (`analysisQuery`, `trendQuery`, `availabilityQuery`, `segmentsQuery`, `segmentActivitiesQuery`), all cached 5 minutes. An `effect()` in the constructor re-`load()`s any already-active query when `FilterStore.fleet()` changes, and a second effect reloads tool-scoped queries when `selectedTool` changes — using `untracked()` so the reload itself doesn't re-trigger the effect.

`alarm.store.ts` — `AlarmStore`: backs the whole Alarm Explorer drill-down (home → fleet → tool → alarm events), one cached `createQuery` per level. Composes `createListFilter` (recipe filter: all/with/without) chained into `createPagination` for the events table, plus `inspectedAlarmId`/`inspectedAlarm` signals driving **the alarm-info <base-drawer> selection** (previously an always-visible inline side panel).

`segment-derivation.util.ts` — pure functions, not a store: `deriveGantt`, `deriveGanttSummary`, `deriveEvents`. Takes the raw `StateSegment[]` feed (+ correlated `SegmentActivity[]`) and derives, respectively, the Gantt chart's per-day system/tool segment bars (with recipe pass/fail detail merged in), a production/downtime summary, and the Event Details table rows. This is the mechanism that keeps the heatmap, Gantt chart, and Event Details table numerically consistent with each other — they're three views over one generated feed, not three independently mocked datasets.

6.5 auth/

- `auth.service.ts` — `AuthService`: signal-based session (`user`, `token`, `isAuthenticated` computed signals). `login(username, password)` checks against `AUTH_CONFIG.demoUsers`, mints a dummy (unsigned, locally-verified-only) JWT-shaped token via base64url-encoded header/payload + a static signature string, and persists `{ token, user }` to `localStorage`. Session is restored from `localStorage` on construction and validated by decoding `exp` from the token payload — no real cryptographic verification (there's no backend to verify against; this is a local-dev stand-in).
- `auth.guard.ts` — `authGuard`/`authChildGuard`: redirect to `/login` with a `returnUrl` query param when `isAuthenticated()` is false. Applied to `ShellComponent`'s route in `app.routes.ts`.
- `auth.interceptor.ts` — `authInterceptor`: attaches `Authorization: Bearer <token>` to every outgoing HTTP request when a token is present.

6.6 constants/

- `app.constants.ts` — the single source of truth for: route path strings (`APP_ROUTES` = absolute paths for routerLink/navigation, `APP_ROUTE_PATHS` = relative segments for the route table), `PLACEHOLDER_ROUTE_PATHS` (the six modules not yet built), `ROUTE_TITLES`, `NAV_GROUPS` (sidebar structure), `APP_BRAND` (name, version, copyright, confidentiality banner), `AUTH_CONFIG` (demo credentials, storage key, token TTL), the topbar's `dateRangeLabel` copy **(NEW)**, the `stateLegend.dayShift` label **(NEW)**, and every other piece of user-facing copy — kept out of templates so copy changes don't touch component logic.
- `component-catalog.ts` — `COMPONENT_CATALOG`: a hand-maintained array (`{ name, selector, group, file, route?, description }`) mirroring every delivered component; the runtime counterpart to this documentation, rendered live at `/dev/components`.

6.7 Theme tokens

Updated this revision: the machine-state palette in `styles.css` (`--color-state-*`) was reassigned to match the current design: Production stays green, Engineering is now blue, Standby is now violet, and Scheduled Downtime is now amber (was: amber, sky-blue, and violet respectively). Unscheduled Downtime and Gap are unchanged. The `.badge-fam` "FAM" tag and the sidebar logo mark both switched from an indigo treatment to a solid emerald green, matching the current brand mark.

7. Layout — The App Shell `layout/`

Not a "feature" — the persistent frame every routed screen renders inside.

- `shell.component.ts` (`fam-shell`) — flex layout: `<fam-sidebar>` + a right column of `<fam-topbar>` , `<router-outlet>` , and a footer (confidentiality banner + session label). Mounted as the component on the root child-route in `app.routes.ts` , guarded by `authGuard` / `authChildGuard` .
- `sidebar.component.ts` (`fam-sidebar`) — dark rail; renders `NAV_GROUPS` from `app.constants.ts` as routerLinks with `routerLinkActive` highlighting. The logo mark is now a solid emerald square **(was an indigo/violet gradient)**.
- `topbar.component.ts` (`fam-topbar`) — `<base-breadcrumbs>` built from the current URL segments (prettified: dashes → spaces, title-case, acronym replacements like "Up Time" → "Up+Time"); **Date Range chip, then** `<base-select>` fleet/duration pickers bound to `FilterStore` ; a user-avatar popup (initials, name, role, sign-out) driven by `AuthService` . Fetches the fleet list once on `ngOnInit` via `ApiService.getFleets()` .

8. Features & Routing `features/`

Screens are intentionally thin: a component injects its store, builds `computed<WidgetConfig[]>`, and renders `<fam-dynamic-page [widgets]="widgets()" />`.

```
/login                                                  LoginComponent (public)

/ (ShellComponent, authGuard)
├── ''                                                   → redirect to /fleet-availability/up-time/analysis
├── fleet-availability/up-time/analysis                 UptimeAnalysisComponent
├── fleet-availability/up-time/availability             UptimeAvailabilityComponent
│       ├── ''                                          → redirect to .../heatmap
│       ├── heatmap                                     StateHeatmapComponent
│       ├── gantt                                        ActivityGanttComponent
│       ├── events                                       EventDetailsComponent
│       └── activities                                   SegmentActivitiesComponent
├── alarm-explorer                                      AlarmHomeComponent
├── alarm-explorer/fleet/:fleetId                        FleetDetailComponent
├── alarm-explorer/fleet/:fleetId/tool/:toolId           ToolAlarmsComponent
├── alarm-explorer/fleet/:fleetId/tool/:toolId/alarm/:alarmId   AlarmEventsComponent
├── fleet-configuration, fleet-productivity, tqual,
│   my-reports, innovation-lab, engineering-utilities     PlaceholderComponent (all six)
├── dev/components                                       ComponentCatalogComponent
└── dev/base                                             BasePlaygroundComponent

**                                                       → redirect to /
```

Route path/title strings all come from `APP_ROUTE_PATHS` / `ROUTE_TITLES` in `core/constants/app.constants.ts` — `app.routes.ts` itself has no hardcoded strings.

8.1 Feature Groups

Area	Files	Store	Notes
Auth	`features/auth/login.component.ts`	AuthService directly	reactive form, base-text-input/base-button/base-alert
Up-Time Analysis	`features/uptime-analysis/uptime-analysis.component.ts`	UptimeStore	Uptime/Downtime toggle trend chart, collapsible breakdown table, top-10 unavailable chart, downtime table, CSV export
Up-Time Availability	`features/uptime-availability/*.component.ts (5 files)`	UptimeStore	page shell + 4 routed tabs + Tool-Level Analysis Filter bar; heatmap/gantt/events share one derived feed
Alarm Explorer	`features/alarm-explorer/*.component.ts (5 files)`	AlarmStore	4-level drill-down (home → fleet → tool → alarm events) + info drawer
Placeholder	`features/placeholder/placeholder.component.ts`	none	"coming soon" screen, reused across 6 unbuilt routes
Dev tools	`features/dev/*.component.ts`	COMPONENT_CATALOG / none	`/dev/components` catalog browser, `/dev/base` live base-module playground

8.2 Component Catalog Detail

Fleet Up+Time Analysis

Route: `/fleet-availability/up-time/analysis`. Mock endpoints: `GET /api/uptime/analysis`, `GET /api/uptime/trend`. Data source: `UptimeStore.analysis()` + `.trend()`. The trend chart offers a Uptime/Downtime toggle; the breakdown table is collapsible with chip-style Tool-wise/ Grouped toggles.

Fleet Up+Time Availability

Component	Selector	Data source	Purpose
UptimeAvailabilityComponent	`fam-uptime-availability`	`UptimeStore.availability()`	Page shell: KPI strip, trend + top-unavailable + downtime-category widgets in one row, the **Tool-Level Analysis Filter** bar (tool picker + state pills), routed tabs
StateHeatmapComponent	`fam-state-heatmap`	`UptimeStore.availability()?.heatmap`	24h × 14-day state heatmap (registered widget: `state-heatmap`)
ActivityGanttComponent	`fam-activity-gantt`	`UptimeStore.gantt / .ganttSummary`	Per-day Sys/Tool state gantt bars, colored availability badges, pagination in the footer (registered widget: `activity-gantt`)
EventDetailsComponent	`fam-event-details`	`UptimeStore.events() / .pagedEvents()`	Table of state-change events, grouped by day (light header style), sortable timestamps, progress-bar duration, delete/edit row actions, paginated (registered widget: `event-details`)
SegmentActivitiesComponent	`fam-segment-activities`	`UptimeStore.segmentActivities() / .pagedSegmentActivities()`	Task/recipe-level activity table correlated with production windows

Route: `/fleet-availability/up-time/availability` with child tabs heatmap / gantt / events / activities. Mock endpoints: `GET /api/uptime/availability`, `GET /api/tools/state-segments`, `GET /api/tools/segment-activities`.

Alarm Explorer

Component	Selector	Route	Purpose
AlarmHomeComponent	`fam-alarm-home`	`/alarm-explorer`	Model/Fleet/Category/Duration filter bar, alarm volume chart (Trend/Pareto tabs), fleet breakdown ranked list → drill into a fleet
FleetDetailComponent	`fam-fleet-detail`	`/alarm-explorer/fleet/:fleetId`	Alarm ID/Tools/Category/SW Version filter bar, KPI strip, tool-level distribution chart, top alarms with category-colored bars, tool summary table → drill into a tool
ToolAlarmsComponent	`fam-tool-alarms`	`/alarm-explorer/fleet/:fleetId/tool/:toolId`	Alarm ID/Category/Severity/Recipe filter bar + searchable/filterable alarm table; row click opens `AlarmInfoPanelComponent` in a `<base-drawer>` → drill into an alarm
AlarmEventsComponent	`fam-alarm-events`	`.../tool/:toolId/alarm/:alarmId`	Date Range/Recipe/Show/Duration filter bar, KPI strip, chronological alarm event log, recipe tab filter, outlined resolution badges, pagination, CSV export
AlarmInfoPanelComponent	`fam-alarm-info-panel`	(rendered inside ToolAlarmsComponent's `<base-drawer>`)	Inspector content for the alarm selected in ToolAlarmsComponent's table: Alarm Info, Occurrence Summary, Recipe Context, and a Weekly Trend chart

Data source: `AlarmStore` throughout. Mock endpoints: `GET /api/alarms/home`, `GET /api/alarms/fleets/:fleetId`, `GET /api/alarms/fleets/:fleetId/tools/:toolId`, `GET /api/alarms/fleets/:fleetId/tools/:toolId/alarms/:alarmId/events`.

Placeholder Modules — Not Yet Built

`PlaceholderComponent` currently serves these routes as a "scaffolded, coming soon" screen — each is the next candidate for component-by-component delivery: Fleet Configuration, Fleet Productivity, TQual, My Reports, Innovation Lab, Engineering Utilities.

9. Reference — Domain Models

See `core/models/models.ts` for the full list. Grouped by feature area:

- **Global:** GlobalFilters, ToolState, AlarmCategory, Severity
- **Up+Time Analysis:** WeeklyUptimePoint, UptimeBreakdownRow, UnavailableTool, DowntimeCategory, UptimeAnalysisResponse, UptimeTrendResponse
- **Up+Time Availability:** AvailabilityKpis, FleetTrendPoint, HeatmapDay, StateTotals, GanttSegment, GanttDay, ToolEvent, AvailabilityResponse **(topUnavailable now includes unscheduledHrs/scheduledHrs/nonScheduledHrs)**, StateSegment, SegmentActivity
- **Alarm Explorer:** FleetAlarmSummary, AlarmVolumeWeek, ToolAlarmSummary, AlarmDefinition, AlarmEvent, AlarmHomeResponse, FleetAlarmDetailResponse **(topAlarms items now include category)**, ToolAlarmDetailResponse, AlarmEventsResponse

10. Adding a New Module — Checklist

Repeatable process for turning a placeholder module — or any new screen — into a real, demoable component backed by mock data:

1. **Model** — add response/row interfaces to `core/models/models.ts`.
2. **Mock data** — add a `build...()` generator to `core/mock-api/mock-data.ts`.
3. **Mock route** — add the matching path branch in `mock-api.interceptor.ts`.
4. **Service** — add a typed method to `ApiService` (`core/services/api.service.ts`).
5. **Store** — add or extend a signal Store under `core/state/`, composing `createQuery` (+ `createPagination` / `createListFilter` if the screen needs paging or client-side filtering).
6. **Component** — build the screen, preferring `WidgetConfig` / `<fam-dynamic-page>` composition over bespoke templates so it matches the rest of the app. Need a bespoke visual (custom chart, grid)? Build it as a normal component, `registerWidget('my-widget', MyComponent)` in `register-widgets.ts`, then reference it as `{ type: 'component', name: 'my-widget' }` — or, if it needs to overlay the whole page (like an inspector), render it directly inside a `<base-drawer>` instead.
7. **Route** — replace the placeholder route entry in `app.routes.ts` with the real `loadComponent`, and update `NAV_GROUPS` / `ROUTE_TITLES` if needed.
8. **Catalog** (optional but expected) — add an entry to `COMPONENT_CATALOG` in `core/constants/component-catalog.ts` so it shows up at `/dev/components`.

This keeps every delivered component demoable standalone (no backend needed) and makes the eventual mock-to-real-API swap a mechanical, low-risk change.